FreshRoute System Design

Project: FreshRoute (Web-based grocery ordering)

Date: March 04, 2026

Prepared by: Team FreshRoute

Location: doc/sprint1/SystemDesign.pdf

This document captures the sprint-1 architecture and design snapshot, including environment assumptions, CRC cards, and error-handling strategy. It is intentionally concise and will evolve in later sprints.

Table of Contents

- 1. Purpose & Scope
- 2. Product Overview
- 3. Technology Stack & Environment
- 4. High-Level Architecture
- 5. System Decomposition
- 6. Data Model
- 7. API Design
- 8. Frontend Components
- 9. CRC Cards
- 10. Error & Exception Strategy
- 11. Assumptions & Constraints

1. Purpose & Scope

Provide a sprint-1 system design snapshot for FreshRoute, covering frontend, backend, and data layers. Align team on responsibilities, interfaces, and dependencies ahead of iterative changes. Meet rubric requirements: cover page, table of contents, CRC cards, architecture diagram, environment description, and error-handling strategy.

2. Product Overview

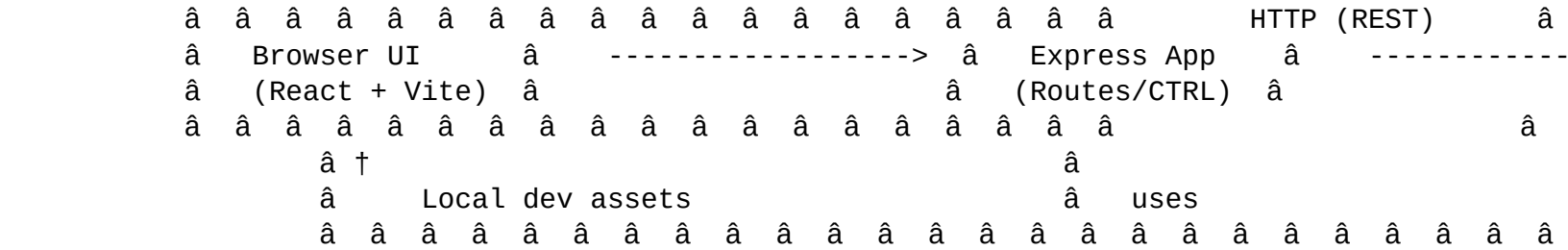
FreshRoute is a web-based platform for browsing organic and ethically raised groceries, shopping curated recipes, managing a cart, and completing orders. The current sprint delivers browsing, search/filter, modal product details, and a basic cart shell.

3. Technology Stack & Environment

- Frontend: React 19 + Vite 7, React-Bootstrap 2, React Icons.
- Backend: Node.js 18+, Express 5, Mongoose 9, CORS, dotenv, Nodemon (dev).
- Database: MongoDB (local Community Server).
- Runtime: Tested locally on macOS/Windows with Node 18+.
- Ports: Backend :4000 (configurable), frontend Vite dev server (usually :5173).
- Environment: backend/.env must define MONGO_URI (and optionally PORT).
- External services: none; assumes local MongoDB reachable from backend host.
- Network: Frontend -> Backend over HTTP REST, CORS enabled for dev.
- Build tools: npm; Vite bundler; ESLint for linting.

4. High-Level Architecture

Pattern: Light MVC on backend (Express routes â controllers â Mongoose models). Frontend uses page-level components plus shared navbar; routing is manual via pathname matching. Deployment model (current sprint): single backend service + SPA frontend served separately during development.



5. System Decomposition

- Client/UI Layer: React pages (Landing, Organic, EthicallyRaised, Recipes, Cart) and shared AppNavbar; handles search, filtering, modals, navigation.
- API Layer: Express routes under /api/products plus health/root; JSON parsing, CORS, product CRUD (create/list implemented).
- Domain/Model Layer: Product Mongoose schema with validation (name, price, category, inStock, timestamps).
- Data Layer: MongoDB collections persisted via Mongoose ODM.
- Configuration: dotenv loads .env; connectDB centralizes Mongo connection; server.js boots app and logs failures.

6. Data Model

Product (MongoDB via Mongoose)

- Fields: name:String! , price:Number! (min 0), category:String (default "Produce"),

inStock:Boolean (default true), timestamps (createdAt/updatedAt).
- Indexes: default _id; no custom indexes yet.
- Validation: required name and price; non-negative price; trims name.

7. API Design

- GET /health â { ok: true, message } (service liveness).
- GET / â plain text "FreshRoute API is running".
- GET /api/products â list products sorted by createdAt desc.
- POST /api/products â create product from request body; 201 on success; 400 on validation error.
- Future: GET by id, PUT/PATCH, DELETE, and category filters to support richer frontend interactions.

8. Frontend Components

- App.jsx: Lightweight router selecting page component by pathname regex.
- AppNavbar: Shared navigation, search or location picker, cart badge, links to main sections.
- Landing: Marketing hero, value props, CTA to Organic page, location picker usage.
- Organic / EthicallyRaised: Tabbed grids for produce/proteins, client-side search/filter, product detail modal, add-to-cart CTA (UI only), membership promo on featured tab.
- Recipes: Tabbed recipe catalog with search, modal showing scaled servings and ingredients, CTA to add servings to cart (UI only).
- Cart: Placeholder shell with navbar; cart logic to be implemented.
- Styling: Shared CSS (Landing.css, Recipes.css, Organic.css, Navbar.css) plus Bootstrap utility classes; assets under src/assets.

9. CRC Cards

Class/Component	Responsibilities	Collaborators	N
App (frontend)	Route to pages by pathname.	Landing, Organic, EthicallyRai	
AppNavbar	Nav links, search/location input, cart badge, welcome text.	All	
Landing	Hero, value props, CTA to shop.	AppNavbar	S
Organic	Tabbed produce grid, search, modal, CTAs.	AppNavbar	
EthicallyRaised	Tabbed protein grid, search, modal.	AppNavbar	
Recipes	Recipe catalog, search, tab filters, modal with servings.	AppNa	
Cart	Cart view shell.	AppNavbar	F
server.js	Load env, connect DB, start HTTP server.	connectDB, app	
app.js	Configure middleware, health/root routes, mount product routes.		
productRoutes	Map /api/products to controllers.	ProductController	
ProductController	Handle product CRUD (create, list).	Product model	
Product model	Define schema, validation, expose queries.	MongoDB	
connectDB	Establish Mongo connection; surface errors.	Mongoose	

10. Error & Exception Strategy

- Backend boot: if MONGO_URI missing or connection fails, server exits with code 1 after logging the error.
- API validation: POST /api/products returns 400 with { error } when Mongoose validation fails.
- Unexpected server errors: can be centralized via middleware; current scope logs to console and returns generic JSON.
- Frontend input: search inputs are local-only now; future API fetch errors should surface toast/error banners.
- Network/DB failures: plan to show user-friendly error states and retry on fetch; backend to set timeouts and respond with 503/504 as needed.
- Data integrity: rely on Mongoose schema; consider unique indexes for SKU/name when inventory expands.

11. Assumptions & Constraints

- MongoDB is reachable locally; no auth in dev. Production will need secure connection strings and secrets management.
- Frontend and backend run on same host during development; CORS open for now (tighten later).
- Cart/checkout flows are placeholders; backend currently only supports product CRUD. Additional services (auth, payments) will be added later.
- Architecture kept simple (single service + SPA) to meet course scope; modularization and routing upgrades deferred.